



Администратор Linux Prometheus



Проверить, идет ли запись

Меня хорошо видно && слышно?



Ставим "+", если все хорошо
"-", если есть проблемы



Тема вебинара

Prometheus



Федоров Иван Романович

Технический директор ГК "Интех"

Опыт:

Более 10 лет в IT-сфере

Кандидат технических наук

Многократный победитель различных конкурсов и хакатонов (команда IBI Solutions)

Эл. почта: ifedorov.devops@gmail.com



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в группе Telegram



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу



Маршрут вебинара



Знакомство

Зачем нужен мониторинг

Prometheus

Grafana

Alertmanager

Рефлексия

Цели вебинара

К концу занятия вы сможете

1. Устанавливать Prometheus



-
2. Ориентироваться в базовой конфигурации



-
3. Создавать dashboard'ы



Цели вебинара

К концу занятия вы сможете

1. Устанавливать Prometheus



2. Ориентироваться в базовой конфигурации

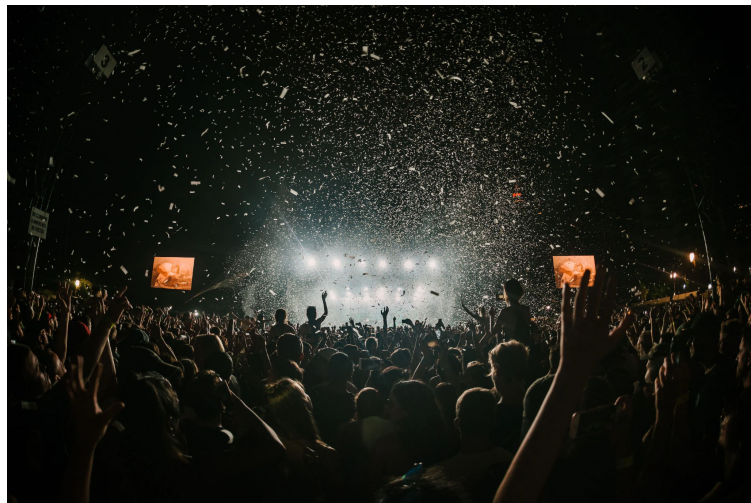


3. Создавать dashboard'ы



Смысл

Зачем вам это уметь



Введение

Ожидания от мониторинга

Почему мы используем мониторинг?

- Оповещения о сбоях в системе => Сбор информации о текущем состоянии
- Предупреждения о возможных сбоях и проблемах
- Сбор статистики и визуализация
- Отчеты
- Красивые дашборды

Blackbox-мониторинг

- Мониторинг, оценивающий внешнее состояние системы/сервиса
 - ping
 - HTTP Request
 - Открытый порт
 - Наличие процессов

Whitebox-мониторинг

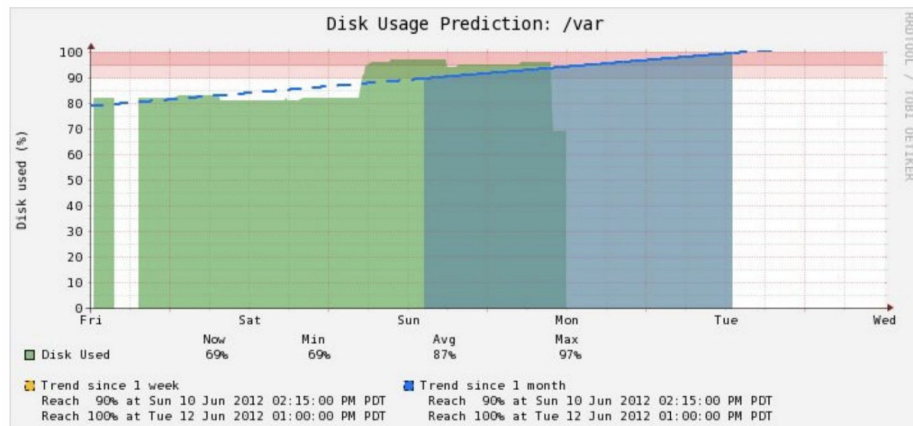
- Мониторинг, базирующийся на метриках, которые дает само приложение (сервис)
 - через логи (журнал протоколирования)
 - интерфейсы
 - API
 - SNMP

Метрики

- Основные метрики системы
 - LoadAvg, CPU, Net (bps/pps), DISK Load
 - Потребление Mem/DISK
 - “Чистота” системных логов (dmesg/messages)
 - Актуальность и состояние резервных копий
- Метрики процесса
 - Наличие процесса и правильное количество этих процессов
 - Открытый сокет/порт процесса
 - Статус процесса (где есть возможность получить)
- Метрики сервиса
 - “Диагностический запрос”, который взаимодействует все или большинство компонентов
 - Время отклика/обработки запроса
 - Количество обращений в единицу времени
 - Количество одновременных обращений

Исторические данные (time series data)

- Что из себя представляют
 - пара дата_время+значение, все это привязано к имени метрики и должно храниться определенное время
- Где их хранить
 - Whisper
 - InfluxDB
 - Prometheus
- Зачем нужны
 - Для анализа проблем
 - Для “гадания” на графиках или trend monitoring



Инструменты мониторинга

- Zabbix
 - свой агент, свой веб-интерфейс, SQL database
- Nagios/icinga
 - свой агент, свой веб-интерфейс, SQL database
- Graphite
 - carbon (агент), whisper (tsdb), graphie-web
- Prometheus
 - node_exporter (агент), prometheus (база, веб), alertmanager
- InfluxData
 - telegraph (агент), influxdb, chronograph (web), capasitor (alerts)
- Grafana
 - интерфейс для дашбордов и алертинга

ZABBIX

Nagios®

**Grafana**

**Prometheus**

Все хорошо? Есть ли вопросы?



Ставим “+”,
если вопросы есть

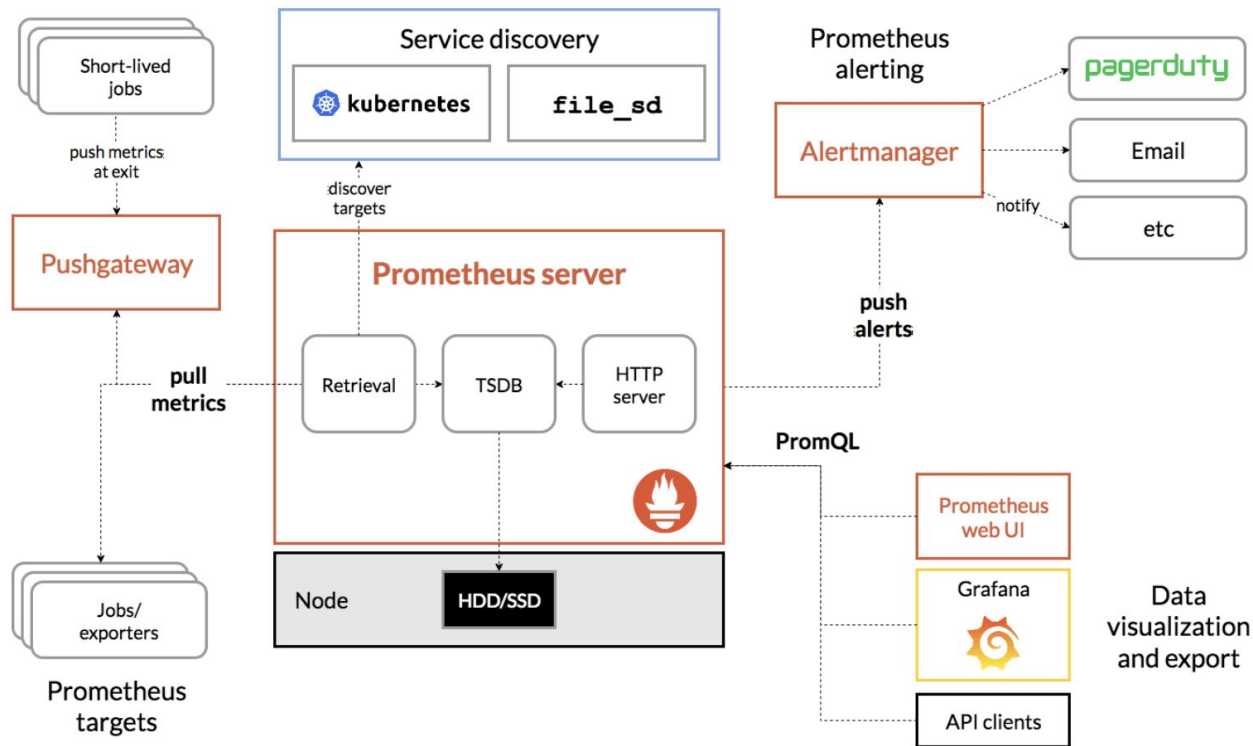


Ставим “-”,
если вопросов нет



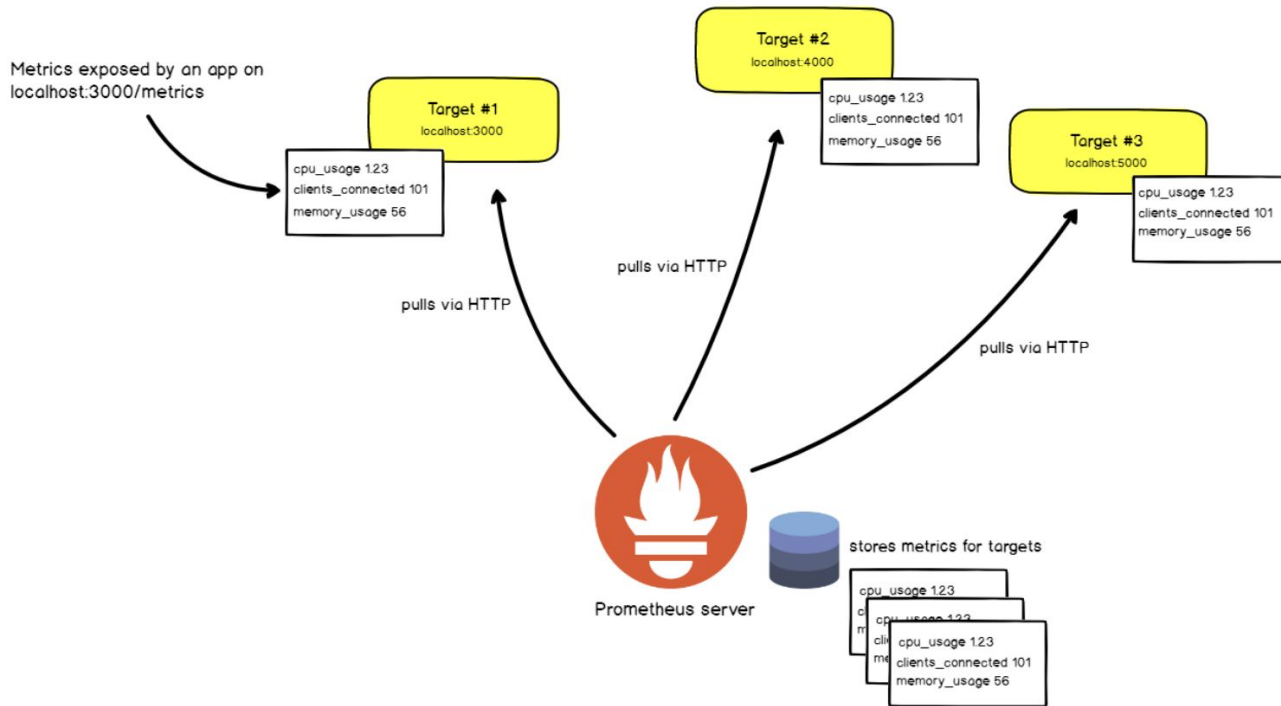
Prometheus

Архитектура



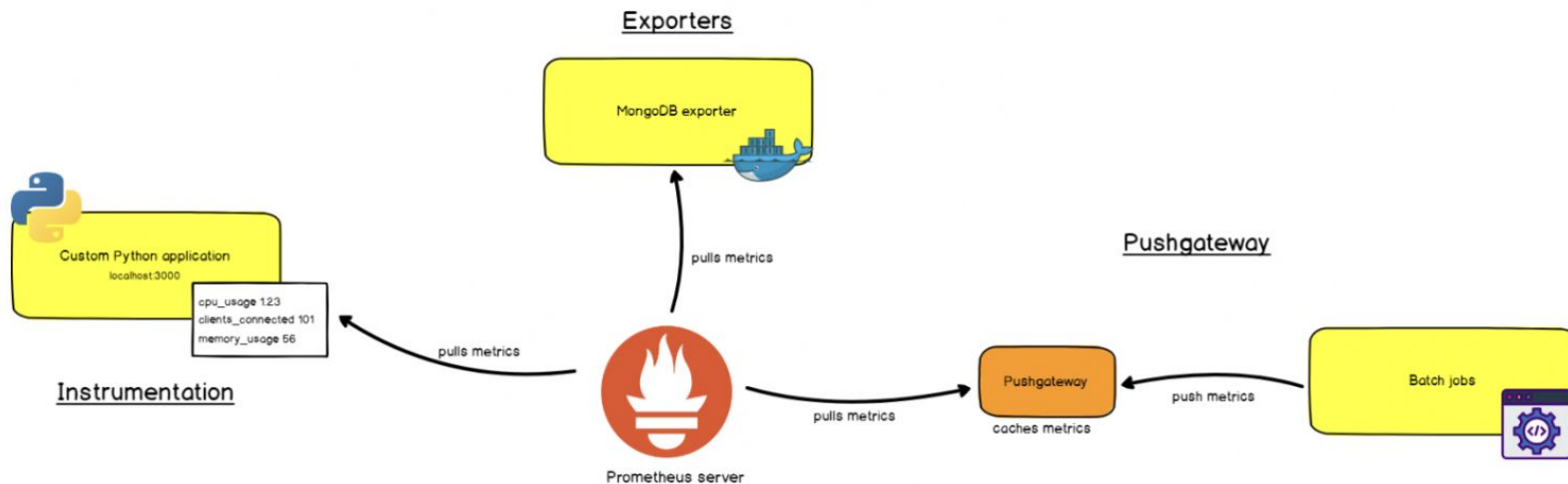
Скрейпинг

What does Prometheus do?



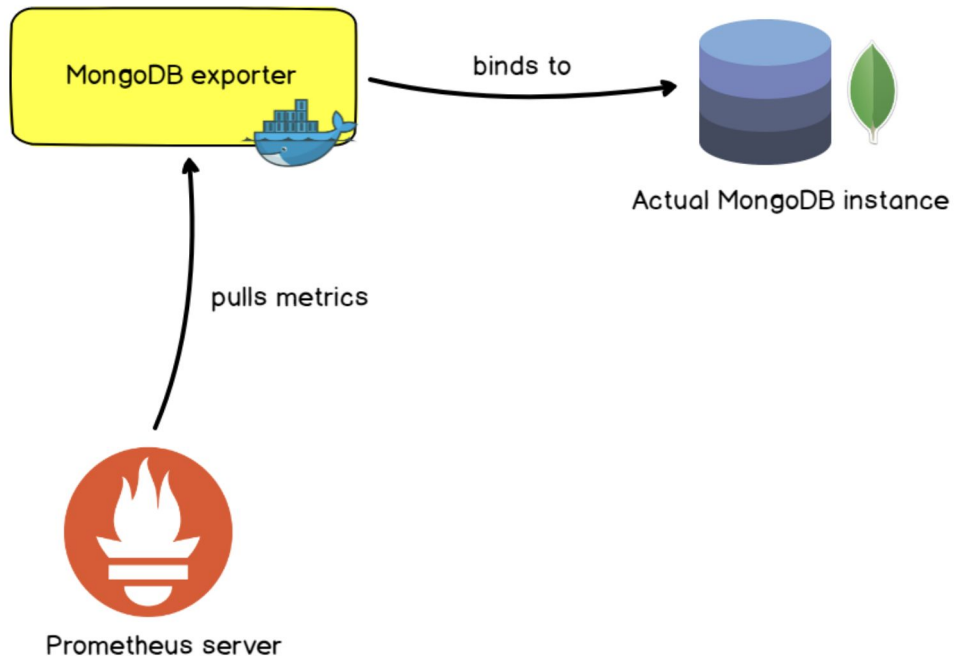
Варианты сбора метрик

Ways to gather metrics in Prometheus

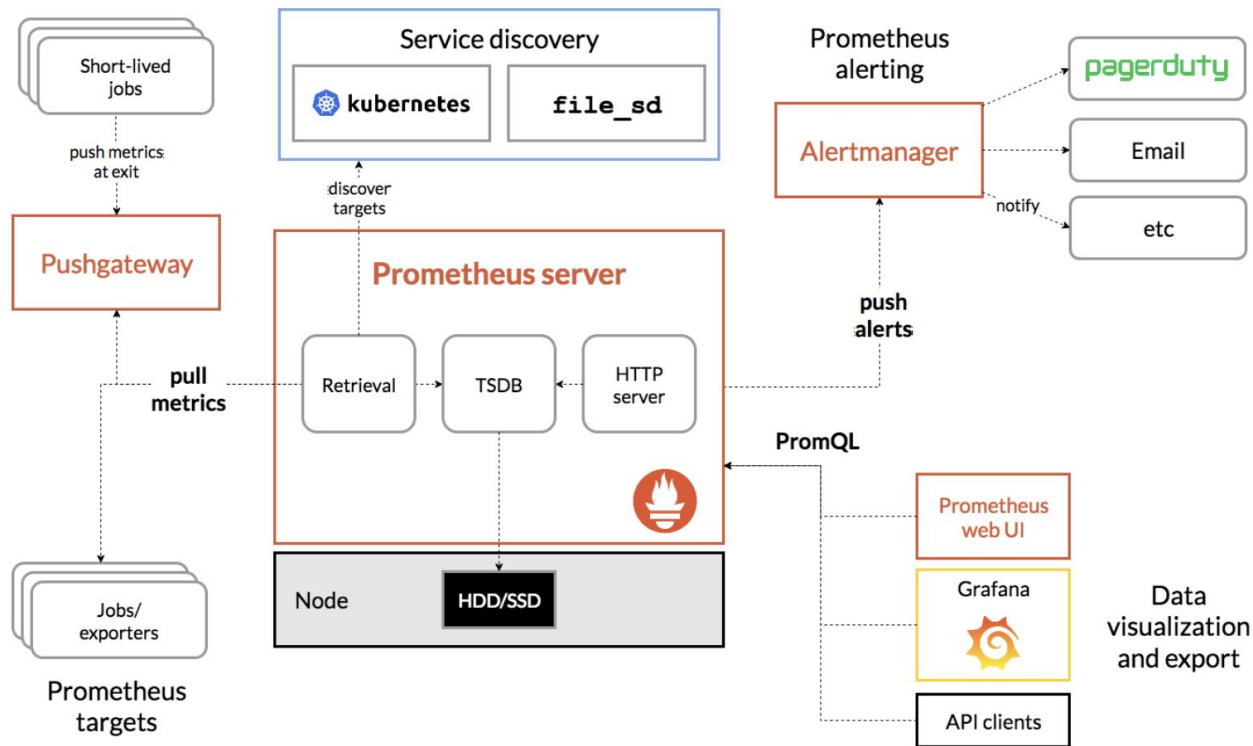


Экспортеры

Exporters with Prometheus



Возвращаемся к архитектуре...



Установка Prometheus (1)

CentOS 7

```
# Устанавливаем вспомогательные пакеты и скачиваем Prometheus
$ yum update -y
$ yum install wget vim -y
$ wget https://github.com/prometheus/prometheus/releases/download/v2.44.0/prometheus-2.44.0.linux-amd64.tar.gz

# Создаем пользователя и нужные каталоги, настраиваем для них владельцев
$ useradd --no-create-home --shell /bin/false prometheus
$ mkdir /etc/prometheus
$ mkdir /var/lib/prometheus
$ chown prometheus:prometheus /etc/prometheus
$ chown prometheus:prometheus /var/lib/prometheus

# Распаковываем архив, для удобства переименовываем директорию и копируем бинарники в /usr/local/bin
$ tar -xvzf prometheus-2.44.0.linux-amd64.tar.gz
$ mv prometheus-2.44.0.linux-amd64 prometheuspackage
$ cp prometheuspackage/prometheus /usr/local/bin/
$ cp prometheuspackage/promtool /usr/local/bin/

# Меняем владельцев у бинарников
$ chown prometheus:prometheus /usr/local/bin/prometheus
$ chown prometheus:prometheus /usr/local/bin/promtool

# По аналогии копируем библиотеки
$ cp -r prometheuspackage/consoles /etc/prometheus
$ cp -r prometheuspackage/console_libraries /etc/prometheus
$ chown -R prometheus:prometheus /etc/prometheus/consoles
$ chown -R prometheus:prometheus /etc/prometheus/console_libraries
```

Установка Prometheus (2)

CentOS 7

```
# Создаем файл конфигурации
$ vim /etc/prometheus/prometheus.yml
global:
  scrape_interval: 10s

scrape_configs:
  - job_name: 'prometheus_master'
    scrape_interval: 5s
    static_configs:
      - targets: ['localhost:9090']

$ chown prometheus:prometheus /etc/prometheus/prometheus.yml
```

Установка Prometheus (3)

CentOS 7

```
# Настраиваем сервис
$ vim /etc/systemd/system/prometheus.service
[Unit]
Description=Prometheus
Wants=network-online.target
After=network-online.target

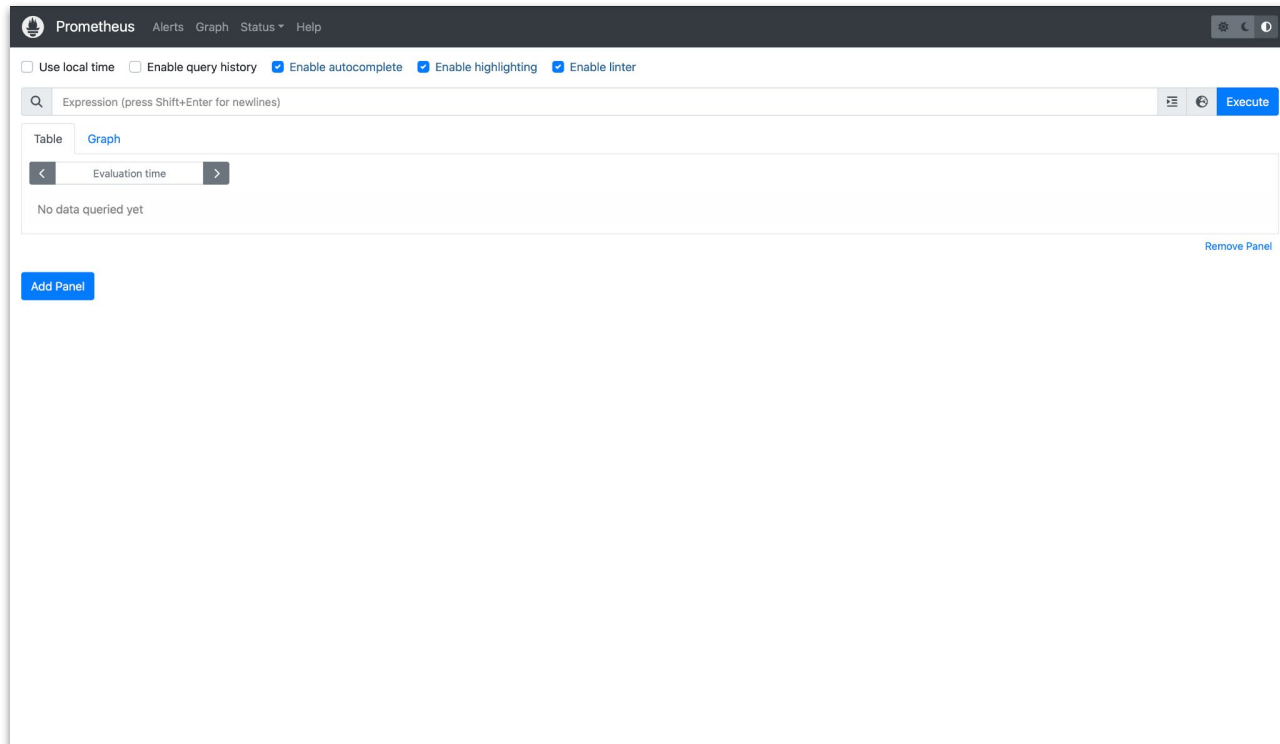
[Service]
User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/prometheus \
--config.file /etc/prometheus/prometheus.yml \
--storage.tsdb.path /var/lib/prometheus/ \
--web.console.templates=/etc/prometheus/consoles \
--web.console.libraries=/etc/prometheus/console_libraries

[Install]
WantedBy=multi-user.target

$ systemctl daemon-reload
$ systemctl start prometheus
$ systemctl status prometheus
```

Проверка

<http://Server-IP:9090/graph>



Установка Node Exporter (1)

CentOS 7

```
# Скачиваем и распаковываем Node Exporter
$ wget https://github.com/prometheus/node_exporter/releases/download/v1.5.0/node_exporter-1.5.0.linux-amd64.tar.gz
$ tar xzfv node_exporter-1.5.0.linux-amd64.tar.gz

# Создаем пользователя, перемещаем бинарник в /usr/local/bin
$ useradd -rs /bin/false nodeusr
$ mv node_exporter-1.5.0.linux-amd64/node_exporter /usr/local/bin/

# Создаем сервис
$ vim /etc/systemd/system/node_exporter.service
[Unit]
Description=Node Exporter
After=network.target

[Service]
User=nodeusr
Group=nodeusr
Type=simple
ExecStart=/usr/local/bin/node_exporter

[Install]
WantedBy=multi-user.target

# Запускаем сервис
$ systemctl daemon-reload
$ systemctl start node_exporter
$ systemctl enable node_exporter
```

Установка Node Exporter (2)

CentOS 7

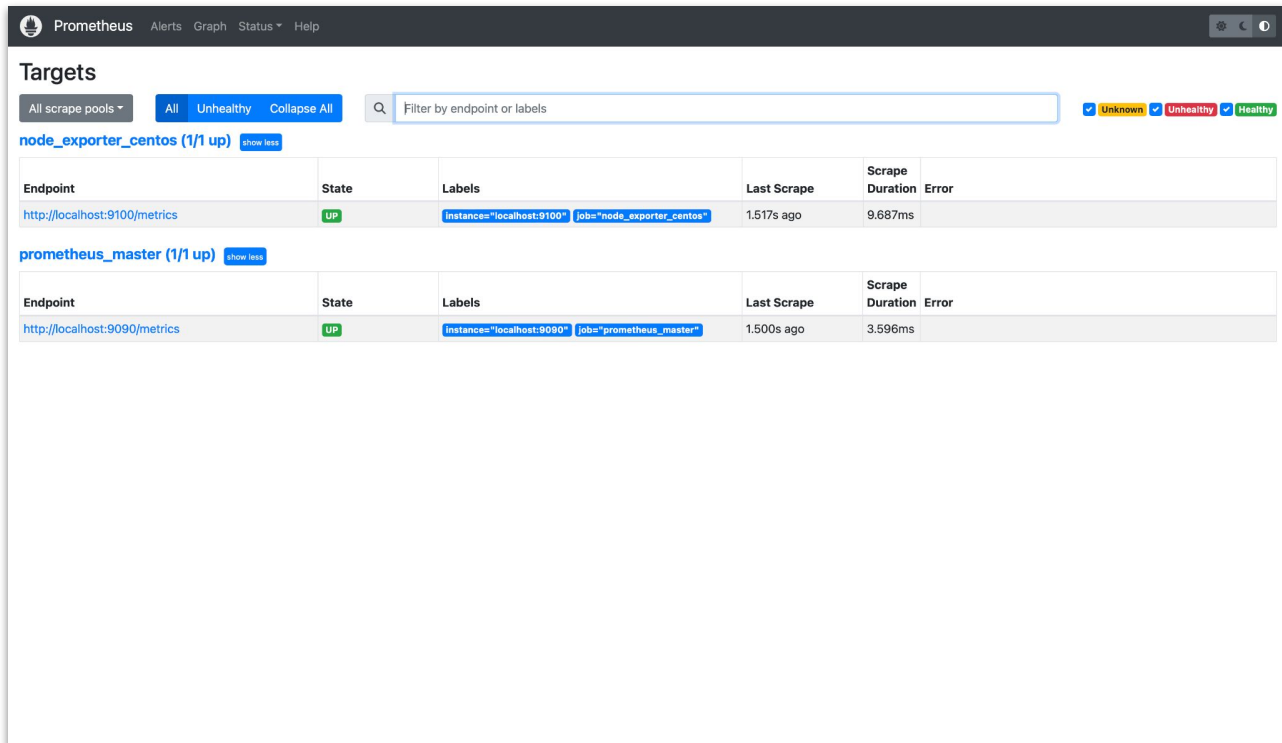
```
# Обновляем конфигурацию Prometheus
$ vim /etc/prometheus/prometheus.yml
global:
  scrape_interval: 10s

scrape_configs:
  - job_name: 'prometheus_master'
    scrape_interval: 5s
    static_configs:
      - targets: ['localhost:9090']
  - job_name: 'node_exporter_centos'
    scrape_interval: 5s
    static_configs:
      - targets: ['localhost:9100']

# Перезапускаем сервис
$ systemctl restart prometheus
```

Проверка 1

<http://Server-IP:9090/targets>



The screenshot displays the Prometheus web interface at the 'Targets' page. The top navigation bar includes 'Prometheus', 'Alerts', 'Graph', 'Status', and 'Help'. Below the navigation bar, there are filters for 'All scrape pools', 'Unhealthy', and 'Collapse All', along with a search bar labeled 'Filter by endpoint or labels'. On the right, there are status indicators for 'Unknown', 'Unhealthy', and 'Healthy'. The main content area shows two target groups: 'node_exporter_centos (1/1 up)' and 'prometheus_master (1/1 up)'. Each group has a 'show less' link. Below each group is a table with columns: Endpoint, State, Labels, Last Scrape, Scrape Duration, and Error.

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9100/metrics	UP	instances="localhost:9100" jobs="node_exporter_centos"	1.517s ago	9.687ms	

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	instances="localhost:9090" jobs="prometheus_master"	1.500s ago	3.596ms	

Проверка 2

<http://NodeExporter-IP:9100/metrics>

```
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 2.6713e-05
go_gc_duration_seconds{quantile="0.25"} 2.9108e-05
go_gc_duration_seconds{quantile="0.5"} 3.1502e-05
go_gc_duration_seconds{quantile="0.75"} 3.7772e-05
go_gc_duration_seconds{quantile="1"} 8.5327e-05
go_gc_duration_seconds_sum 0.006286368
go_gc_duration_seconds_count 182
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 10
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.19.3"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated and still in use.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 2.270136e+06
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated, even if freed.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 3.16286552e+08
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.512499e+06
# HELP go_memstats_frees_total Total number of frees.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 4.270287e+06
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 8.415056e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and still in use.
# TYPE go_memstats_heap_alloc_bytes gauge
go_memstats_heap_alloc_bytes 2.270136e+06
# HELP go_memstats_heap_idle_bytes Number of heap bytes waiting to be used.
# TYPE go_memstats_heap_idle_bytes gauge
go_memstats_heap_idle_bytes 4.38272e+06
# HELP go_memstats_heap_inuse_bytes Number of heap bytes that are in use.
# TYPE go_memstats_heap_inuse_bytes gauge
go_memstats_heap_inuse_bytes 3.64544e+06
# HELP go_memstats_heap_objects Number of allocated objects.
# TYPE go_memstats_heap_objects gauge
go_memstats_heap_objects 14590
# HELP go_memstats_heap_released_bytes Number of heap bytes released to OS.
# TYPE go_memstats_heap_released_bytes gauge
go_memstats_heap_released_bytes 3.145728e+06
# HELP go_memstats_heap_sys_bytes Number of heap bytes obtained from system.
# TYPE go_memstats_heap_sys_bytes gauge
go_memstats_heap_sys_bytes 8.02816e+06
# HELP go_memstats_last_gc_time_seconds Number of seconds since 1970 of last garbage collection.
# TYPE go_memstats_last_gc_time_seconds gauge
go_memstats_last_gc_time_seconds 1.6842426393839808e+09
# HELP go_memstats_lookups_total Total number of pointer lookups.
# TYPE go_memstats_lookups_total counter
go_memstats_lookups_total 0
# HELP go_memstats_mallocs_total Total number of mallocs.
# TYPE go_memstats_mallocs_total counter
go_memstats_mallocs_total 4.284877e+06
# HELP go_memstats_mcache_inuse_bytes Number of bytes in use by mcache structures.
# TYPE go_memstats_mcache_inuse_bytes gauge
go_memstats_mcache_inuse_bytes 1200
# HELP go_memstats_mcache_sys_bytes Number of bytes used for mcache structures obtained from system.
# TYPE go_memstats_mcache_sys_bytes gauge
go_memstats_mcache_sys_bytes 15600
# HELP go_memstats_mspan_inuse_bytes Number of bytes in use by mspan structures.
# TYPE go_memstats_mspan_inuse_bytes gauge
go_memstats_mspan_inuse_bytes 53178
```

Как настроение? Есть ли вопросы?



Ставим “+”,
если вопросы есть

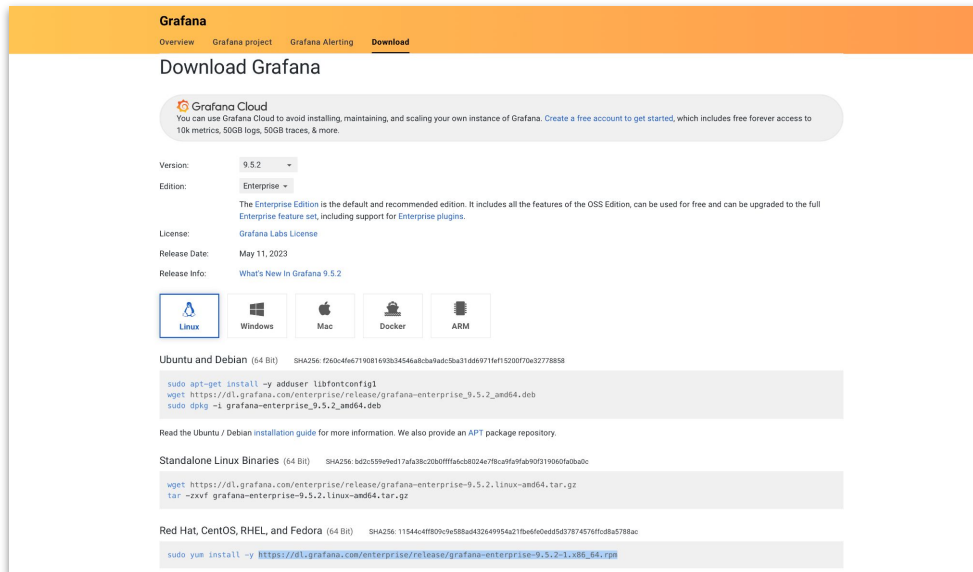


Ставим “-”,
если вопросов нет

Grafana

Проблема

Не доступна в нашем регионе :(



Sorry, our service is currently not available in your region

Решение

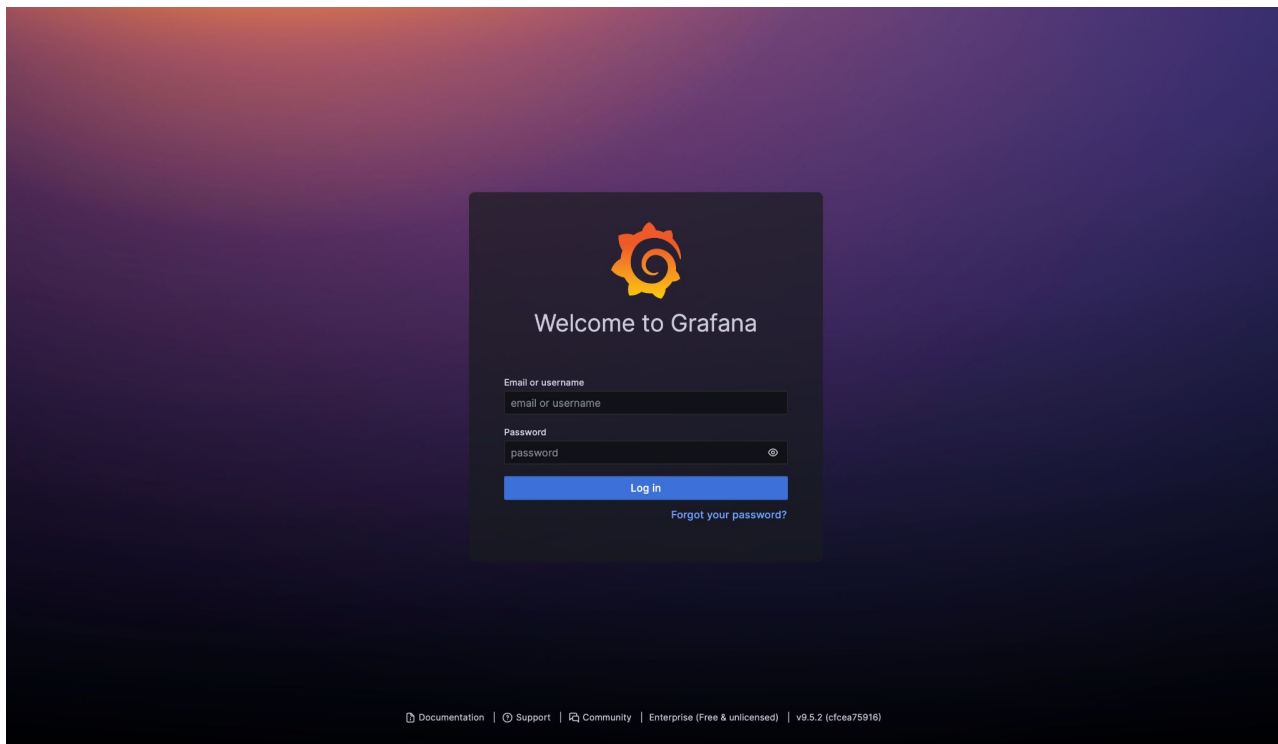
CentOS 7

```
# Скачать grafana-enterprise-9.5.2-1.x86_64.rpm через VPN и поместить на сервер
$ yum -y install grafana-enterprise-9.5.2-1.x86_64.rpm

# Стартуем сервис
$ systemctl daemon-reload
$ systemctl start grafana-server
```

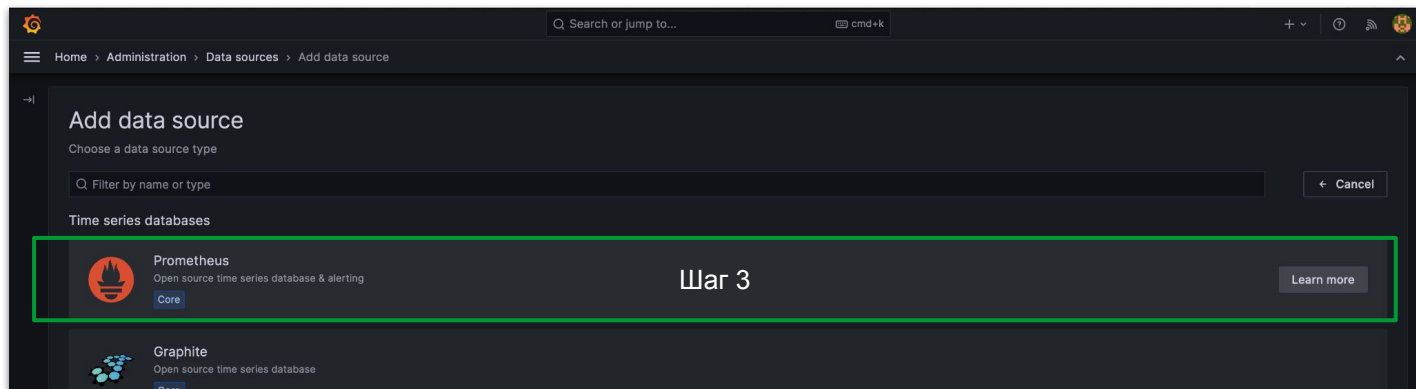
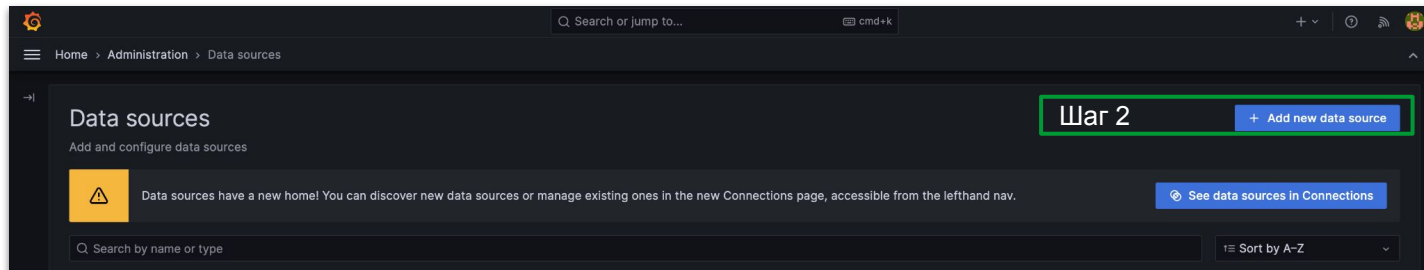
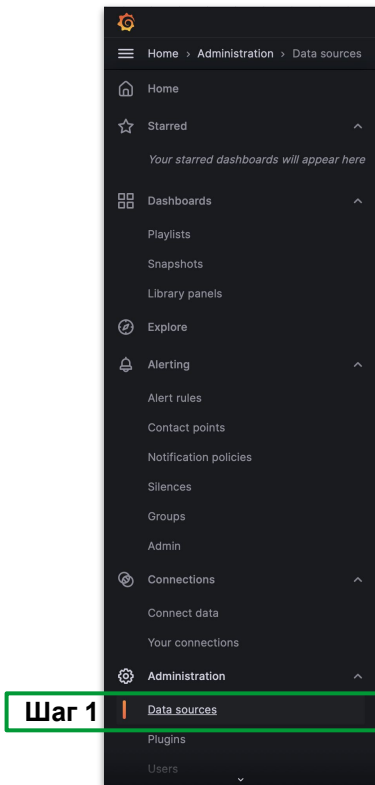
Проверка 1

<http://Server-IP:3000> (admin/admin)



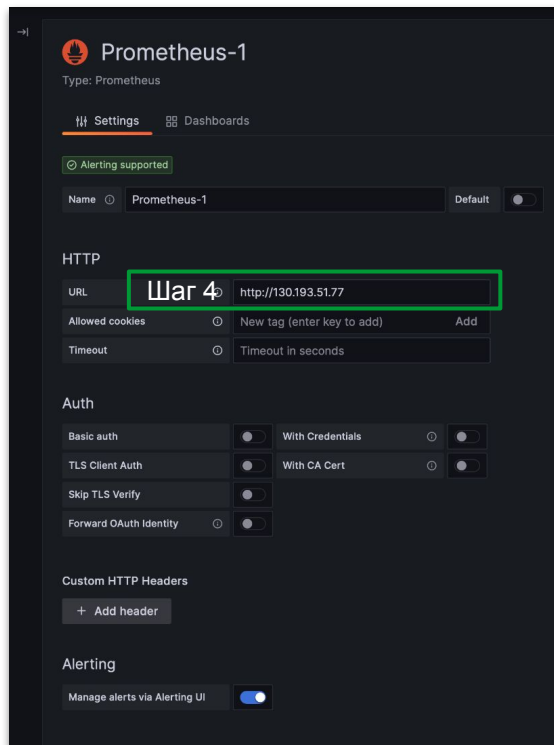
Интеграция с Prometheus (1)

Добавление источника данных



Интеграция с Prometheus (2)

Добавление источника данных



Prometheus-1
Type: Prometheus

Settings Dashboards

Alerting supported

Name Prometheus-1 Default

HTTP

URL **Шар 4** http://130.193.51.77

Allowed cookies New tag (enter key to add) Add

Timeout Timeout in seconds

Auth

Basic auth With Credentials

TLS Client Auth With CA Cert

Skip TLS Verify

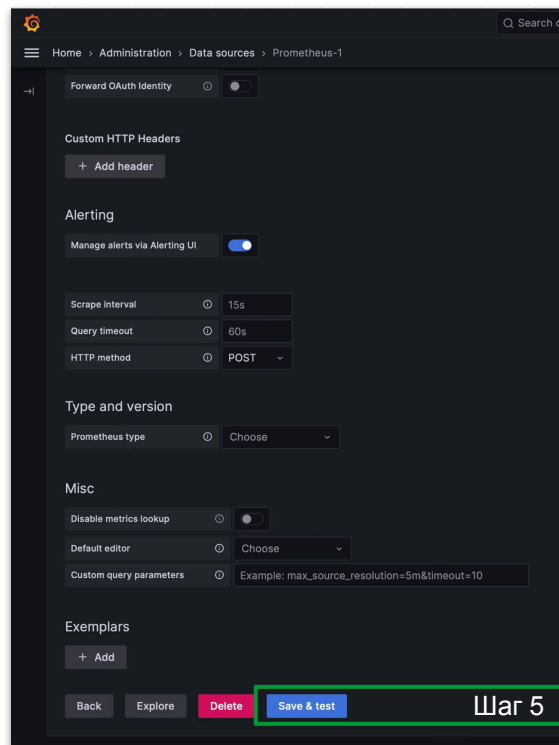
Forward OAuth Identity

Custom HTTP Headers

+ Add header

Alerting

Manage alerts via Alerting UI



Home Administration Data sources Prometheus-1

Forward OAuth Identity

Custom HTTP Headers

+ Add header

Alerting

Manage alerts via Alerting UI

Scrape interval 15s

Query timeout 60s

HTTP method POST

Type and version

Prometheus type Choose

Misc

Disable metrics lookup

Default editor Choose

Custom query parameters Example: max_source_resolution=5m&timeout=10

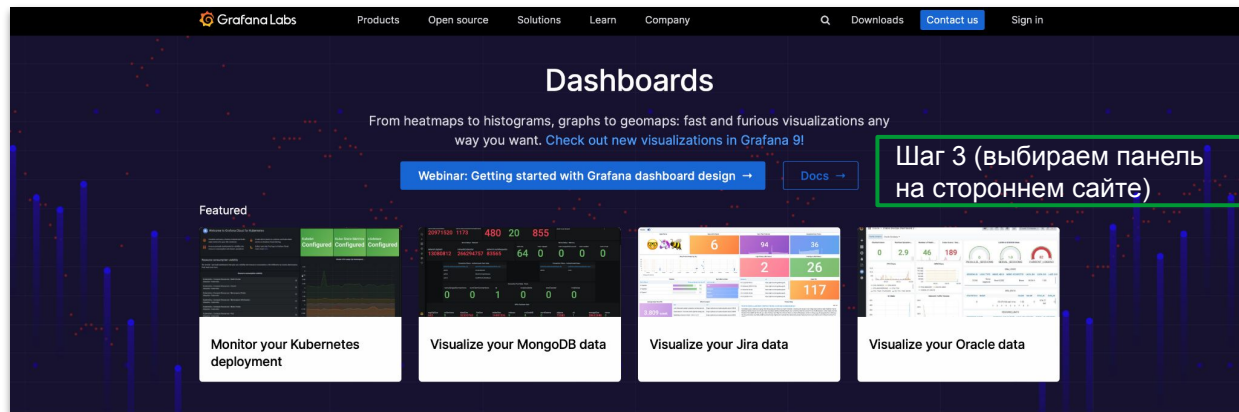
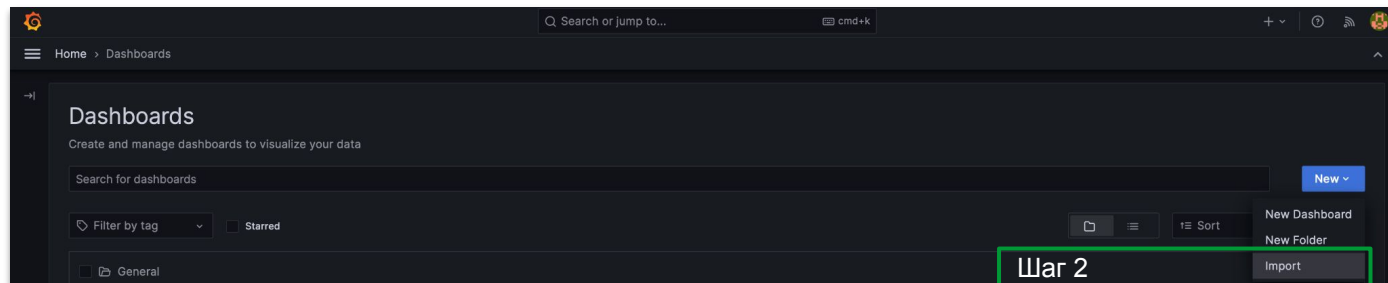
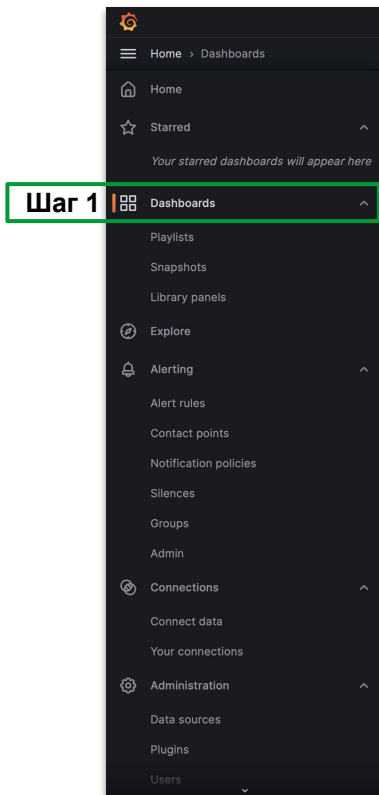
Exemplars

+ Add

Back Explore Delete **Save & test** Шар 5

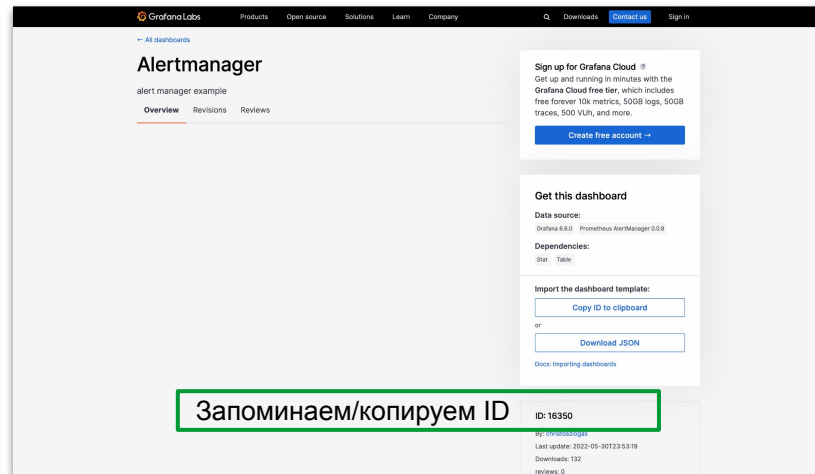
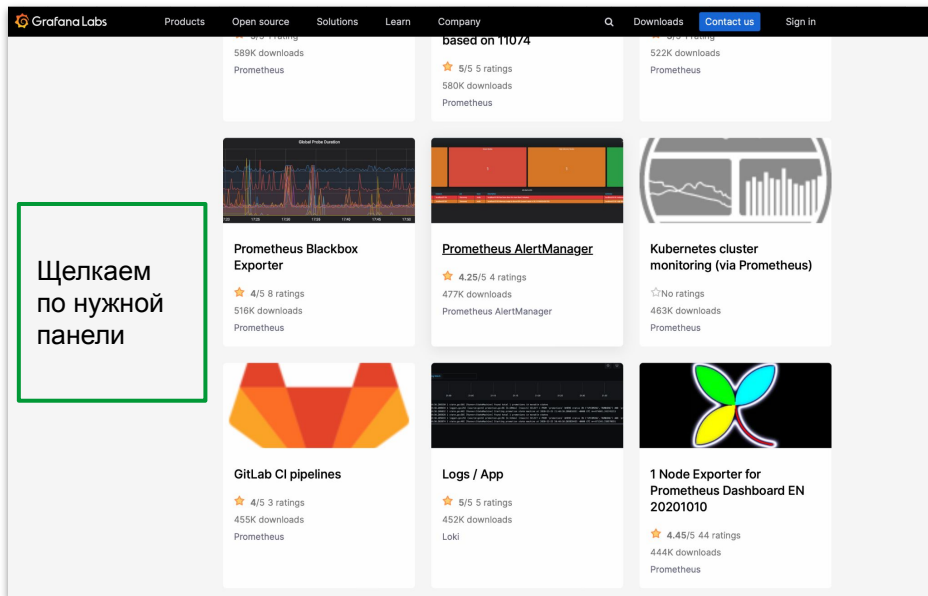
Создание Dashboard (1)

Импорт панели из <https://grafana.com/grafana/dashboards/>



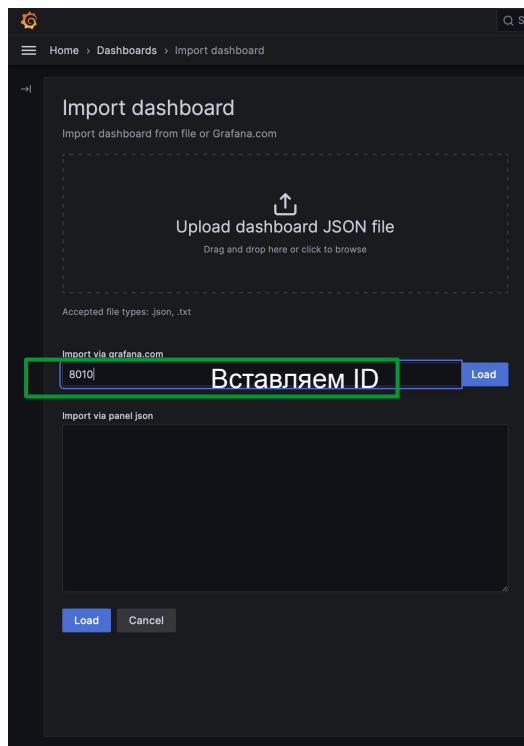
Создание Dashboard (2)

Импорт панели из <https://grafana.com/grafana/dashboards/>



Создание Dashboard (3)

Импорт панели из <https://grafana.com/grafana/dashboards/>



Import dashboard

Import dashboard from file or Grafana.com

Upload dashboard JSON file

Drag and drop here or click to browse

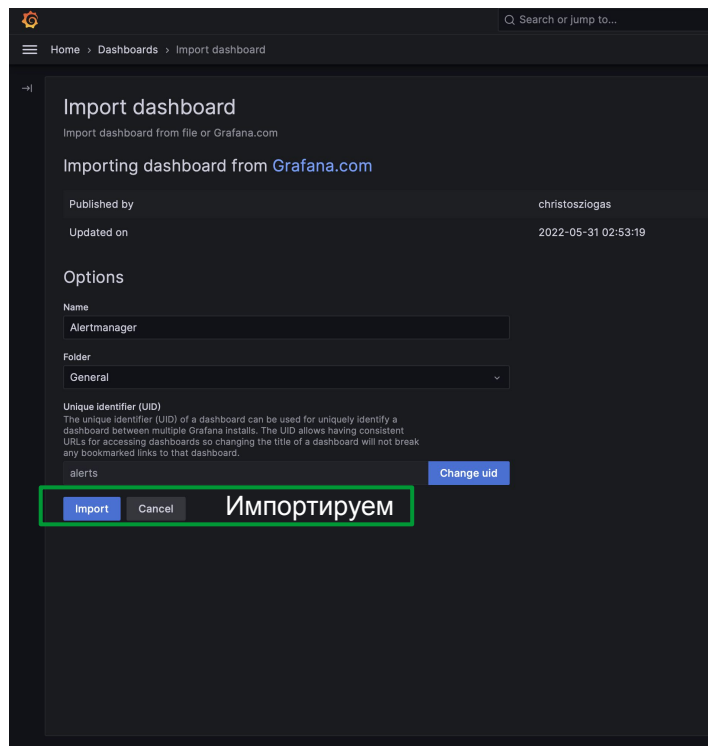
Accepted file types: .json, .txt

Import via grafana.com

8010 Вставляем ID Load

Import via panel json

Load Cancel



Import dashboard

Import dashboard from file or Grafana.com

Importing dashboard from Grafana.com

Published by christosziogas

Updated on 2022-05-31 02:53:19

Options

Name Alertmanager

Folder General

Unique identifier (UID)

The unique identifier (UID) of a dashboard can be used to uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.

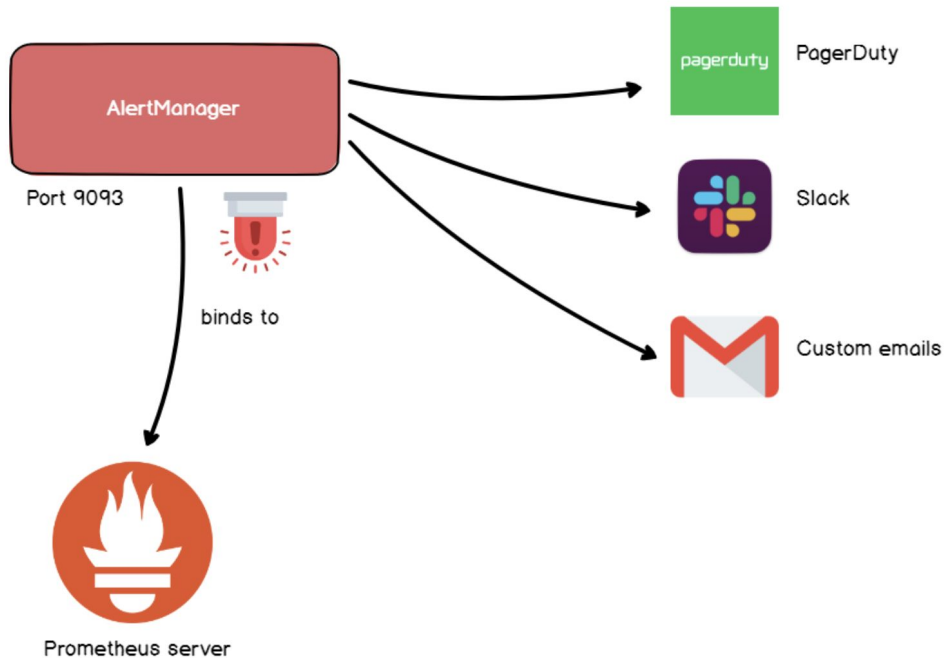
alerts Change uid

Import Cancel Импортируем

AlertManager

Алертинг

Alerting with Prometheus



Установка AlertManager (1)

CentOS 7

```
# Скачиваем и распаковываем AlertManager
$ wget https://github.com/prometheus/alertmanager/releases/download/v0.25.0/alertmanager-0.25.0.linux-amd64.tar.gz
$ tar xzf alertmanager-0.25.0.linux-amd64.tar.gz

# Создаем пользователя и нужные директории
$ useradd --no-create-home --shell /bin/false alertmanager
$ usermod --home /var/lib/alertmanager alertmanager
$ mkdir /etc/alertmanager
$ mkdir /var/lib/alertmanager

# Копируем бинарники из архива в /usr/local/bin и меняем владельца
$ cp alertmanager-0.25.0.linux-amd64/amtool /usr/local/bin/
$ cp alertmanager-0.25.0.linux-amd64/alertmanager /usr/local/bin/
$ cp alertmanager-0.25.0.linux-amd64/alertmanager.yml /etc/alertmanager/
$ chown -R alertmanager:alertmanager /etc/alertmanager /var/lib/alertmanager
$ chown alertmanager:alertmanager /usr/local/bin/{alertmanager,amtool}
$ echo "ALERTMANAGER_OPTS=\"\" > /etc/default/alertmanager
$ chown alertmanager:alertmanager /etc/default/alertmanager
$ chown -R alertmanager:alertmanager /var/lib/prometheus/alertmanager
```

Установка AlertManager (2)

CentOS 7

```
# Настраиваем сервис
$ vim /etc/systemd/system/alertmanager.service
[Unit]
Description=Alertmanager Service
After=network.target prometheus.service

[Service]
EnvironmentFile=-/etc/default/alertmanager
User=alertmanager
Group=alertmanager
Type=simple
ExecStart=/usr/local/bin/alertmanager \
    --config.file=/etc/alertmanager/alertmanager.yml \
    --storage.path=/var/lib/prometheus/alertmanager \
    $ALERTMANAGER_OPTS
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
Restart=always

[Install]
WantedBy=multi-user.target

# Запускаем сервис
$ systemctl daemon-reload
$ systemctl start alertmanager
```

Установка AlertManager (3)

CentOS 7

```
# Настраиваем правила
$ vim /etc/prometheus/rules.yml
groups:
- name: alert.rules
  rules:
  - alert: InstanceDown
    expr: up == 0
    for: 1m
    labels:
      severity: critical
    annotations:
      description: '{{ $labels.instance }} of job {{ $labels.job }} has been down for more than 1 minute.'
      summary: Instance {{ $labels.instance }} down

# Проверяем валидность
$ /usr/local/bin/promtool check rules /etc/prometheus/rules.yml
```



Установка AlertManager (4)

CentOS 7

```
# Обновляем конфиг prometheus
$ vim /etc/prometheus/prometheus.yml
global:
  scrape_interval: 10s

scrape_configs:
  - job_name: 'prometheus_master'
    scrape_interval: 5s
    static_configs:
      - targets: ['localhost:9090']
  - job_name: 'node_exporter_centos'
    scrape_interval: 5s
    static_configs:
      - targets: ['localhost:9100']

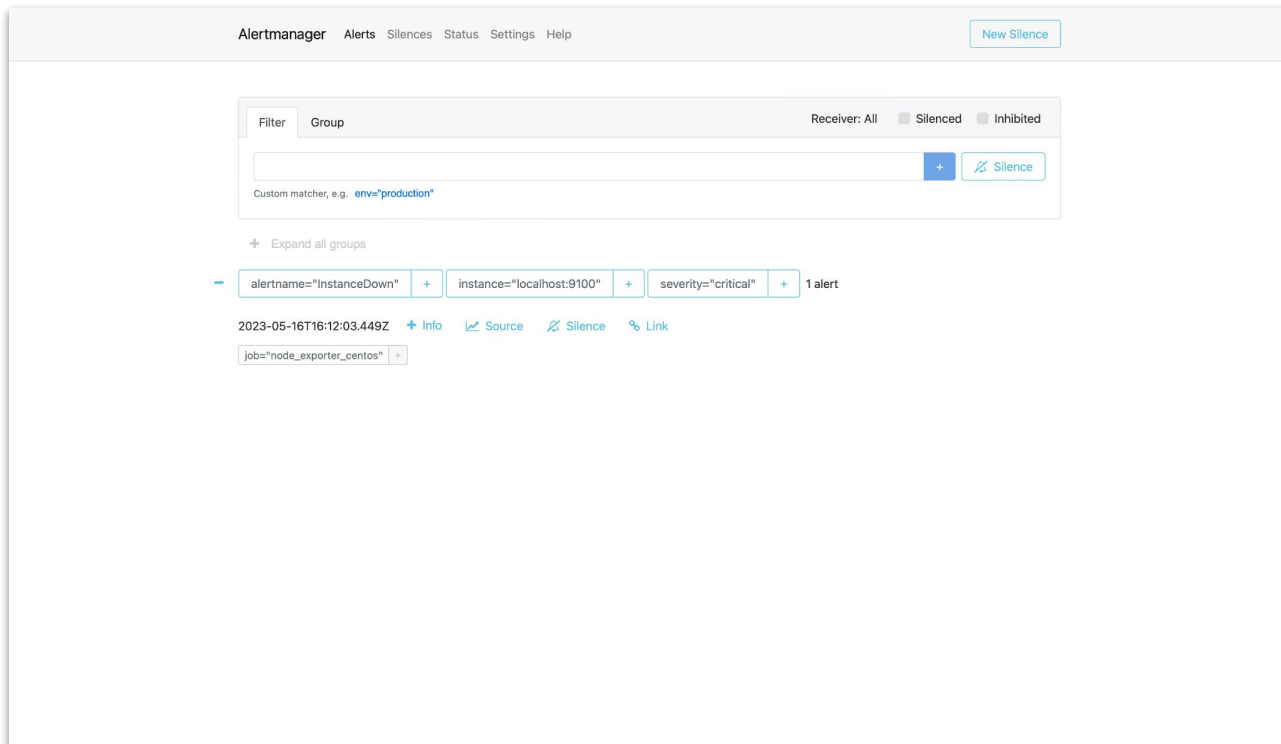
rule_files:
  - "rules.yml"

alerting:
  alertmanagers:
    - static_configs:
      - targets:
        - localhost:9093

# Рестартуем сервисы
$ systemctl restart prometheus
$ systemctl restart alertmanager
```

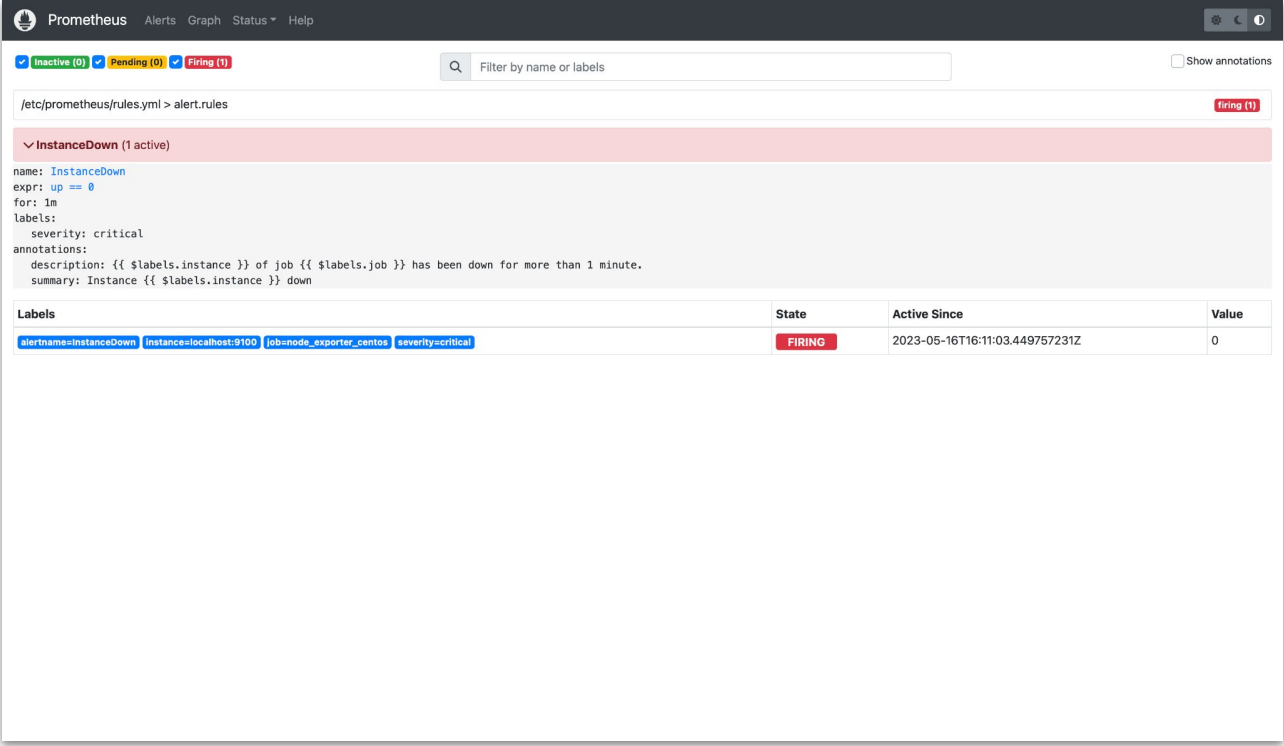
Проверка 1

<http://Server-IP:9093>



Проверка 2

<http://Server-IP:9090>



The screenshot shows the Prometheus Alerts interface. At the top, there are tabs for Alerts, Graph, Status, and Help. Below the tabs, there are filters for Inactive (0), Pending (0), and Firing (1). A search bar is available with the placeholder text "Filter by name or labels". The main content area shows the alert rules configuration for `/etc/prometheus/rules.yml > alert.rules`. The `InstanceDown` alert is active, with the following configuration:

```
name: InstanceDown
expr: up == 0
for: 1m
labels:
  severity: critical
annotations:
  description: '{{ $labels.instance }} of job {{ $labels.job }} has been down for more than 1 minute.
  summary: Instance {{ $labels.instance }} down
```

Below the configuration, there is a table showing the alert details:

Labels	State	Active Since	Value
<code>alertname=instanceDown</code> <code>instance=localhost:9100</code> <code>job=node_exporter_centos</code> <code>severity=critical</code>	FIRING	2023-05-16T16:11:03.449757231Z	0

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет

Вопросы для проверки

Какой компонент Prometheus используется для сбора метрик с целевых систем (например, операционной системы или приложений)?

- A. Alertmanager
- B. Grafana
- C. Prometheus (TSDB)
- D. Экспортеры (Exporters)



Отправьте в чат верную букву

Вопросы для проверки

Какой компонент Prometheus используется для сбора метрик с целевых систем (например, операционной системы или приложений)?

A. Alertmanager

B. Grafana

C. Prometheus (TSDB)

D. Экспортеры (Exporters)

Вопросы для проверки

Какой параметр в глобальной секции конфигурации Prometheus (prometheus.yml) задает стандартный интервал времени для сбора метрик (скрейпинга)?

- A. job_name
- B. targets
- C. rule_files
- D. scrape_interval



Отправьте в чат верную букву

Вопросы для проверки

Какой параметр в глобальной секции конфигурации Prometheus (prometheus.yml) задает стандартный интервал времени для сбора метрик (скрейпинга)?

A. job_name

B. targets

C. rule_files

D. scrape_interval

Вопросы для проверки

Какой тип мониторинга оценивает внешнее состояние системы/сервиса, используя такие методы, как ping, HTTP-запросы или проверку открытых портов?

- A. Whitebox-мониторинг
- B. Blackbox-мониторинг
- C. Мониторинг процесса
- D. Мониторинг сервиса



Отправьте в чат верную букву

Вопросы для проверки

Какой тип мониторинга оценивает внешнее состояние системы/сервиса, используя такие методы, как ping, HTTP-запросы или проверку открытых портов?

A. Whitebox-мониторинг

B. Blackbox-мониторинг

C. Мониторинг процесса

D. Мониторинг сервиса

Вопросы для проверки

Какой инструмент используется для создания дашбордов и визуализации данных, полученных Prometheus, а также для настройки алертинга?

- A. Alertmanager
- B. Zabbix
- C. Graphite
- D. Grafana



Отправьте в чат верную букву

Вопросы для проверки

Какой инструмент используется для создания дашбордов и визуализации данных, полученных Prometheus, а также для настройки алертинга?

- A. Alertmanager
- B. Zabbix
- C. Graphite
- D. Grafana**

Вопросы для проверки

В какой формат данных Prometheus сохраняет исторические данные (time series data), который характеризуется парой дата_время + значение, привязанной к имени метрики?

- A. Логи (журналы протоколирования)
- B. SNMP (Simple Network Management Protocol)
- C. Исторические данные (Time Series Data)
- D. Отчеты (Reports)



Отправьте в чат верную букву

Вопросы для проверки

В какой формат данных Prometheus сохраняет исторические данные (time series data), который характеризуется парой дата_время + значение, привязанной к имени метрики?

- A. Логи (журналы протоколирования)
- B. SNMP (Simple Network Management Protocol)
- C. Исторические данные (Time Series Data)**
- D. Отчеты (Reports)

Рефлексия

Цели вебинара

Проверка достижения целей

1. Установили Prometheus



-
2. Разобрались в базовой конфигурации



-
3. Научились создавать dashboard'ы



Рефлексия



С какими впечатлениями уходите с вебинара?



Как будете применять на практике то, что узнали на вебинаре?

**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**